

JavaScript Event Loop – Complete Theory Notes

1. What is the Event Loop?

JavaScript is a **single-threaded** language — it can execute **one task at a time**.

However, browsers (and Node.js) allow JavaScript to handle asynchronous tasks like:

- `setTimeout()`
- Promises
- Fetch API (network calls)
- Event listeners
- Database operations (in Node)
- File system operations (in Node)

The **Event Loop** is the mechanism that enables JavaScript to perform **non-blocking asynchronous operations** even though it has a single thread.

2. Components of JavaScript Runtime

A JavaScript runtime (like Chrome V8 or Node.js) contains:

✓ 1. Call Stack

Where function execution happens.

Works on principle: **LIFO (Last In, First Out)**.

✓ 2. Web APIs (Browser APIs)

Provided by the browser, not JavaScript.

Examples:

- `setTimeout()`
- DOM events
- HTTP requests
- Geolocation API

✓ 3. Callback Queue / Task Queue (Macro Task Queue)

Stores callbacks of:

- `setTimeout`
- `setInterval`
- DOM events
- I/O tasks

These run **after** the call stack is empty.

✓ 4. Microtask Queue

Contains microtasks like:

- Promise callbacks (`.then()`)

- MutationObserver
- async/await (internally uses promises)

Microtasks have **higher priority** than macrotasks.

✓ 5. Event Loop

The manager that keeps checking:

If the Call Stack is empty → push tasks from queues to the stack.

3. How the Event Loop Works (Step-by-Step)

Step 1: Execute synchronous code first

Everything inside the call stack is run first.

Step 2: If an async operation begins, send it to Web APIs

Example:

```
setTimeout(() => console.log("Timer done"), 2000);
```

Timer runs inside Web API (browser), NOT in the call stack.

Step 3: When the async task completes, push its callback to a queue

- setTimeout → goes to callback queue (macrotask queue)
- Promise.then → goes to microtask queue

Step 4: When call stack is empty, Event Loop checks queues

Microtasks run BEFORE macrotasks.

4. Microtask Queue vs Macrotask Queue

Macrotasks (Task Queue)

Examples:

- setTimeout
- setInterval
- I/O operations
- UI events
- setImmediate (Node.js)

Microtasks (Higher Priority)

Examples:

- Promise.then()

- async/await
- queueMicrotask()
- MutationObserver

Execution Order

1. Execute all synchronous code
2. Execute all microtasks (until queue is empty)
3. Take one macrotask and run it
4. Repeat microtasks
5. Take next macrotask...

Microtasks always drain fully before moving to the macrotask queue.

5. Example: Promises vs setTimeout

```
console.log("Start"); setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise")); console.log("End");
```

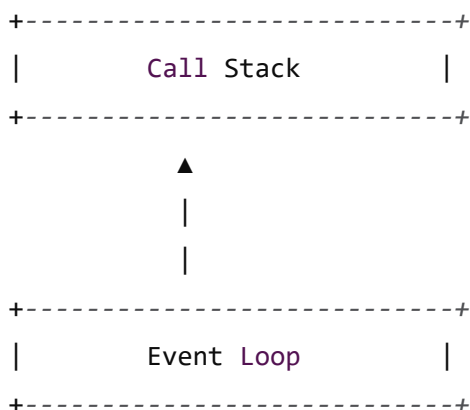
Output:

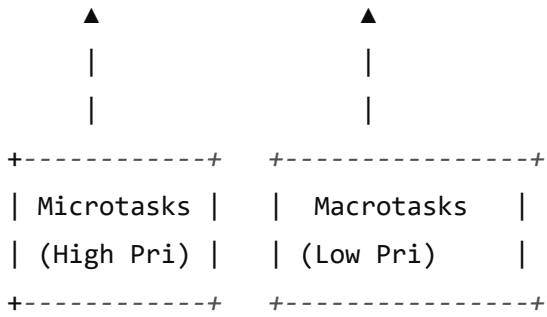
```
Start
End
Promise
Timeout
```

Why?

- setTimeout → macrotask queue
 - Promise → microtask queue (higher priority)
-

6. Visual Event Loop Flow





7. Real Example With Deep Understanding

```
console.log("1"); setTimeout(() => console.log("2"), 0); Promise.resolve().then(() => console.log("3")).then(() => console.log("4")); console.log("5");
```

Output:

```
1
5
3
4
2
```

Explanation:

1 → synchronous
5 → synchronous
3 → microtask
4 → microtask
2 → macrotask

8. Event Loop in async/await

```
async function demo() { console.log("A"); await Promise.resolve(); console.log("B"); }  
demo(); console.log("C");
```

Output:

```
A
C
```

Why?

- `await` pauses only inside async function
 - After `await`, the continuation goes to the **microtask queue**
-

9. Node.js Event Loop (Extra Knowledge)

Node uses **libuv** and has **six phases**:

1. **Timers Phase**
2. **Pending Callbacks Phase**
3. **Idle / Prepare Phase**
4. **Poll Phase (I/O)**
5. **Check Phase**
6. **Close Callbacks Phase**

Node also prioritizes **microtasks** after every phase.

10. Why the Event Loop is Important?

- ✓ Avoid blocking UI
 - ✓ Understand async behavior
 - ✓ Prevent callback hell
 - ✓ Debug timing issues
 - ✓ Master performance optimization
 - ✓ Essential for interviews (Frontend + Node.js)
 - ✓ Helps understand promises, `async/await`, callbacks deeply
-

11. Quick Summary Table

Concept	Description
Call Stack	Executes JS synchronously

Concept	Description
Web APIs	Browser handles async tasks
Microtask Queue	Promise callbacks; highest priority
Macrotask Queue	Timers, events, network tasks
Event Loop	Pushes tasks to stack when it's empty
Priority	Microtasks > Macrotasks